

第2章 文本数据处理——从字符串到模型输入

大模型的输入是文本，怎么进行训练和推理？

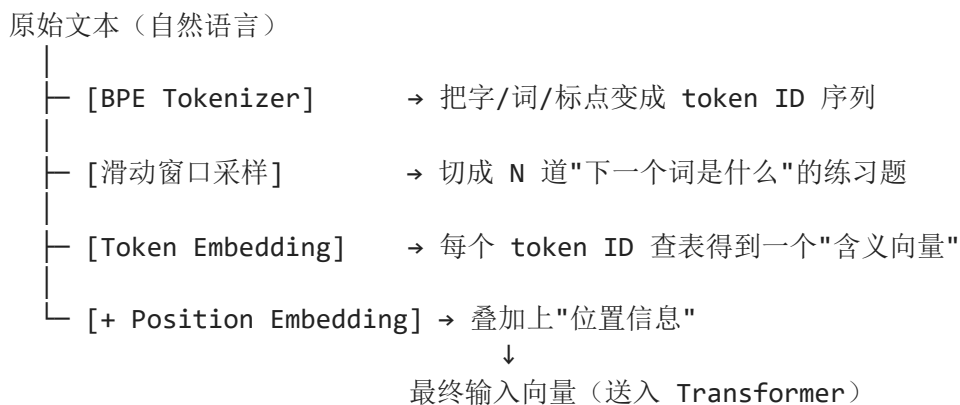
一个很自然的问题：我们平时看到的文字，模型是怎么“读懂”的？它只是一堆矩阵运算，不认识字，只认识**数字**。所以 Chapter 2 只做一件事：**把自然语言一步步变成模型能计算的数字**。

怎么变？分四步走，从笨办法一路升级到工业方案：

1. **自己写个最简陋的分词器（文本->数字转换工具）** → 理解“词表”和“token ID”到底是什么
2. **换成 BPE 子词分词** → 解决“词表里没有的词怎么办”（GPT-2 的方案）
3. **用滑动窗口切出训练样本** → 每道题都是“看到前面，猜下一个”
4. **token ID → 向量 + 位置编码** → 变成模型真正能吃的输入

下面一步步来。

完整数据流（学完本章回头看这张图就够）



第一步：加载数据

先把原材料准备好。本章用到的文本是短篇小说《The Verdict》大概 2 万个字符，不大不小，刚好练手。

下面把必要的库装上，下载文本，看一眼前 99 个字符长什么样：

```
In [1]: from importlib.metadata import version
        from pathlib import Path

        print("torch version:", version("torch"))
        print("tiktoken version:", version("tiktoken"))

        import requests
```

```

# 兼容两种常见运行方式:
# 1. 在项目根目录运行 notebook: data/the-verdict.txt
# 2. 在 notebooks/ 目录运行 notebook: ../data/the-verdict.txt
DATA_PATH = next(
    (path for path in [Path("data/the-verdict.txt"), Path("../data/the-verdict.t
    Path("../data/the-verdict.txt"),
)
DATA_PATH.parent.mkdir(parents=True, exist_ok=True)

if not DATA_PATH.exists():
    url = (
        "https://raw.githubusercontent.com/rasbt/"
        "LLMs-from-scratch/main/ch02/01_main-chapter-code/"
        "the-verdict.txt"
    )
    response = requests.get(url, timeout=30)
    response.raise_for_status()
    DATA_PATH.write_bytes(response.content)

# 导入原始文本
raw_text = DATA_PATH.read_text(encoding="utf-8")

print("Total number of character:", len(raw_text))
print(raw_text[:99])

```

torch version: 2.12.0

tiktoken version: 0.13.0

Total number of character: 20479

I HAD always thought Jack Gisburn rather a cheap genius--though a good fellow eno
ugh--so it was no

下面开始最核心的一步：**把文本切成 token，然后给每个 token 编个号。**

为什么需要编号？因为模型只认数字。比如 "Hello world" 这句话，模型需要的不是 H-e-l-l-o 这些字母，而是类似 [3412, 789] 的整数序列。所以我们必须先建一张"单词 → 编号"的对照表，也就是**词表 (vocabulary)**。

先从最简单的切分方式开始——按空格和标点符号来切：

In [2]: # 第一次尝试：只按空格切分

```

import re

text = "Hello, world. This, is a test."
result = re.split(r'(\s)', text)
print(result)

```

```
['Hello,', ' ', 'world.', ' ', 'This,', ' ', 'is', ' ', 'a', ' ', 'test.']
```

标点粘在词上了——Hello, 和 world. 被当成了一个整体，逗号句号没单独拆出来。这不行，标点应该独立成 token。

In [3]: # 第二次尝试：加上逗号和句号作为分隔符

```

result = re.split(r'([.,]\s)', text)
print(result)

```

```
['Hello', ',', ' ', 'world', '.', ' ', 'This', ',', ' ', 'is', ' ', 'a', ' ', 'test', '.', ' ']
```

标点被拆出来了！但冒出来一堆空字符串 '' —— 这是因为逗号后面紧跟着空格，两个分隔符之间什么都没来，就生成了空串。

```
In [4]: # 第三次尝试：过滤掉空字符串
result = [item.strip() for item in result if item.strip()]
print(result)

['Hello', ',', 'world', '.', 'This', ',', 'is', 'a', 'test', '.']
```

干净了！但现在只处理了逗号和句号——还有问号、分号、引号、括号、双破折号等等没管呢。

```
In [5]: # 第四次尝试：补全所有常见标点，这就是最终版的正则了
text = "Hello, world. Is this-- a test?"
result = re.split(r'([,.;?_!"()\'']|--|\s)', text)
result = [item.strip() for item in result if item.strip()]
print(result)

['Hello', ',', 'world', '.', 'Is', 'this', '--', 'a', 'test', '?']
```

```
In [6]: # 使用正则表达式将文本分割为单词
import re

text = raw_text
# 正则解释：按以下规则切分—
#  [,.;?_!"()\''] → 标点符号单独切开
#  -- → 双破折号单独切开
#  \s → 空格/换行作为分隔
#  括号把分隔符也保留下来（否则会被丢掉）
result = re.split(r'([,.;?_!"()\'']|--|\s)', text)
result = [item.strip() for item in result if item.strip()]
print(result[:10])
# 得到result为含有单词和标点的列表，接下来就可以根据这些单词和标点，生成一份简单的
print("文本所含标点与单词总数：", len(result))

# 去重、排序
all_words = sorted(set(result))
vocab_size = len(all_words)
print("去重后所含单词和标点的数目：", vocab_size)
# 将单词对应序号
vocab = {token:integer for integer,token in enumerate(all_words)}
for i, item in enumerate(vocab.items()):
    print(item)
    if i >= 5:
        break
```

```
['I', 'HAD', 'always', 'thought', 'Jack', 'Gisburn', 'rather', 'a', 'cheap', 'gen
ius']
```

文本所含标点与单词总数： 4690

去重后所含单词和标点的数目： 1130

```
('!', 0)
(',', 1)
('.', 2)
('(', 3)
(')', 4)
(',', 5)
```

这样，词表就有了——1130 个不重复的 token，每个都分配了一个整数 ID。

有了词表，就可以做两件事：

- **encode**: 文本 → token ID 序列（喂给模型）
- **decode**: token ID 序列 → 文本（把模型输出翻译回人话）

我们把这两个操作封装成一个类，方便复用：

```
In [7]: class SimpleTokenizerV1:
    def __init__(self, vocab: dict):
        # 输入是一个字典，就是我们上文中生成过的 {token: token_id} 的字典
        self.str_to_int = vocab
        self.int_to_str = {i:s for s, i in vocab.items()}

        # 所谓的encoder就是将字符串转为token_id的过程
    def encode(self, text: str) -> list[int]:
        preprocessed = re.split(r'([,.;?_"()\' ]|--|\s)', text)

        preprocessed = [
            item.strip() for item in preprocessed if item.strip()
        ]
        ids = [self.str_to_int[s] for s in preprocessed]
        return ids

    # decoder就是将token_id 转文本
    def decode(self, ids) -> str:
        text = " ".join([self.int_to_str[i] for i in ids])
        # Replace spaces before the specified punctuations
        text = re.sub(r'\s+([,.;?_"()\' ])', r'\1', text)
        return text

    # 示例用法（实例化方式）：
tokenizer = SimpleTokenizerV1(vocab)
text = """It's the last he painted, you know,"
        Mrs. Gisburn said with pardonable pride."""
ids = tokenizer.encode(text)
print(ids)
tokenizer.decode(ids)
```

```
[1, 56, 2, 850, 988, 602, 533, 746, 5, 1126, 596, 5, 1, 67, 7, 38, 851, 1108, 75
4, 793, 7]
```

```
Out[7]: '" It\' s the last he painted, you know," Mrs. Gisburn said with pardonable pri
de.'
```

注意：这个简陋 tokenizer 已经能工作，但它会把 It's 拆得很怪。这正是我们后面需要 BPE 的原因之一。

我们现在写的分词器能将单词转为 token ID——前提是这个词在词表里。
V1 的词表只覆盖了《The Verdict》这本小说里出现的 1130 个词，换一段新文本，大概率会碰到生词。怎么办？

最简单的思路：**不认识就老实说“不认识”**。在词表里塞几个“占位符”（特殊 token），让分词器碰到生词时用它代替，而不是直接报错。

常见的特殊 token 有：

- `<|unk|>`：未知词 (unknown)，替换词表里没有的词
- `<|endoftext|>`：文本结束标记，用来分隔两篇来源不同的文章（比如训练数据里有 Wikipedia + 小说，中间就用它隔开）；GPT-2 还顺手拿它做 padding
- `[BOS]` / `[EOS]` / `[PAD]`：有些模型还会加"句首" "句尾" "填充" 标记

⚠ GPT-2 实际上只用 `<|endoftext|>` 一个特殊 token——它不需要 `<|unk|>`（因为 BPE 能把生词拆成已知子词，第二阶段会讲），也不单独区分 BOS/EOS/PAD，全部复用 `<|endoftext|>`。这里是教学路线，我们先从"加占位符"的方案走一遍，理解动机后再看 GPT-2 的最终方案。

下面先用 V1 分词器试试，看它碰到生词会怎样：

```
In [8]: tokenizer = SimpleTokenizerV1(vocab)

text = "Hello, do you like tea. Is this-- a test?"

tokenizer.encode(text)
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[8], line 5
      1 tokenizer = SimpleTokenizerV1(vocab)
      2
      3 text = "Hello, do you like tea. Is this-- a test?"
      4
----> 5 tokenizer.encode(text)

Cell In[7], line 14, in SimpleTokenizerV1.encode(self, text)
     10
     11     preprocessed = [
     12         item.strip() for item in preprocessed if item.strip()
     13     ]
--> 14     ids = [self.str_to_int[s] for s in preprocessed]
     15     return ids

KeyError: 'Hello'
```

可以看到，词典里没有 "Hello" 这个词，也没有 `<|unk|>` 这个占位符，所以 V1 直接报 `KeyError` 了。先把 `<|endoftext|>` 和 `<|unk|>` 插入词典：

```
In [9]: all_tokens = sorted(list(set(result)))
all_tokens.extend(["<|endoftext|>", "<|unk|>"])

vocab = {token:integer for integer,token in enumerate(all_tokens)}
len(vocab.items())
for i, item in enumerate(list(vocab.items())[-5:]):
    print(item)
# 成功将两个标记插入到词典

('younger', 1127)
('your', 1128)
('yourself', 1129)
('<|endoftext|>', 1130)
('<|unk|>', 1131)
```

但我们刚刚写的Tokenizer还不能处理这些特殊的符号（它不知道这些符号是干什么的，需要我们告诉它怎么处理），接下来写新版的Tokenizer：

```
In [10]: class SimpleTokenizerV2:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = { i:s for s,i in vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([,.;?!"()\' ]|--|\s)', text)
        preprocessed = [item.strip() for item in preprocessed if item.strip()]
        preprocessed = [
            item if item in self.str_to_int
            else "<|unk|>" for item in preprocessed # 让tokenizer将不认识的单词
        ]

        ids = [self.str_to_int[s] for s in preprocessed]
        return ids

    def decode(self, ids):
        text = " ".join([self.int_to_str[i] for i in ids])
        # Replace spaces before the specified punctuations
        text = re.sub(r'\s+([,.;?!"()\' ])', r'\1', text)
        return text

tokenizer = SimpleTokenizerV2(vocab)

text1 = "Hello, do you like tea?"
text2 = "In the sunlit terraces of the palace."

text = " <|endoftext|> ".join((text1, text2))

print(text)
```

Hello, do you like tea? <|endoftext|> In the sunlit terraces of the palace.

```
In [11]: # 试试encode是否正常
tokenizer.encode(text)
```

```
Out[11]: [1131, 5, 355, 1126, 628, 975, 10, 1130, 55, 988, 956, 984, 722, 988, 1131, 7]
```

```
In [12]: # 再试试将encode后的tokenid列表再decode
tokenizer.decode(tokenizer.encode(text))
```

```
Out[12]: '<|unk|>, do you like tea? <|endoftext|> In the sunlit terraces of the <|unk|>.'
```

可以看到，不认识的单词都被标记成了 <|unk|>，至此，Tokenizer已经可以处理任何文本了（只是可能出现一些不认识的单词）

那还有什么问题呢？还需要优化哪些地方？

当然有问题，而且不小：

1. 不认识的字都标记成 <|unk|> ——模型的知识盲区会很大。一段正常文本里如果有好几个 <|unk|>，模型等于在“盲猜”，生成质量可想而知。

2. **词表会无限膨胀**——世界上的语言、人名、新造词无穷无尽，靠"完整单词进词表"这条路，词表只会越来越大，而且永远追不上新词出现的速度。

下面来实际感受一下，用一段含生词的文本跑 V2：

```
In [ ]: # 模拟一段包含生僻词的文本——比如医学、科幻、或者纯粹瞎编的词
text_with_rare_words = (
    "The doctor diagnosed pneumonoultramicroscopicsilicovolcanoconiosis. "
    "Meanwhile, the cyborgs from Zeta-Reticuli were deploying their"
    "quasifibrillators near the heliopause. "
    "Jasper's selfbuilt tokenizer definitely cannot handle this."
)

ids_v2 = tokenizer.encode(text_with_rare_words)
decoded_v2 = tokenizer.decode(ids_v2)

print("原文：")
print(text_with_rare_words)
print("\nV2 encode → decode 结果：")
print(decoded_v2)

# 统计一下 <|unk|> 的数量
unk_count = sum(1 for i in ids_v2 if tokenizer.int_to_str[i] == "<|unk|>")
total_count = len(ids_v2)
print(f"\n总共 {total_count} 个 token，其中 <|unk|> 占了 {unk_count} 个 ({unk_co
```

一句话里一半都是 <|unk|>，模型基本在盲猜。这就是"完整单词进词表"的死穴——换一个领域、换一种语言，词表就得跟着膨胀，永远追不上。

怎么解决？核心思路：**不要把"完整单词"当作最小单位。把生词拆成更小的已知片段，让模型用这些片段去拼。**比如 `selfbuilt` 拆成 `self` + `built`，`quasifibrillators` 拆成 `quasi` + `fibr` + `ill` + `ators`。

这就是 BPE (Byte Pair Encoding，字节对编码) 做的事。下面来正式认识它。

BPE 不一定把一个英文单词当成一个 token。它可以把生词拆成更小的已知片段，所以不需要把每个完整单词都放进词表。

```
In [13]: # 注意：前面写 V1、V2 是为了理解分词器的原理。
# 实际项目中不会自己从零写——直接用成熟的 BPE 分词器即可。
# 下面 tokenizer 使用 GPT-2 的 BPE 分词器。

import importlib
import tiktoken # OpenAI 开源的 BPE 分词器，Rust 实现，速度约为纯 Python 的 5 倍

print("tiktoken version:", importlib.metadata.version("tiktoken"))
tokenizer = tiktoken.get_encoding("gpt2") # ← 从这里开始，tokenizer 是 BPE 分词
```

tiktoken version: 0.13.0

试试它的效果

```
In [14]: text = (
    "Hello, do you like tea? <|endoftext|> In the sunlit terraces"
    "of someunknownPlace."
)

# BPE 分词器默认会把 <|endoftext|> 当作普通文本处理（拆成 <、|、end 等子词）。
# 加上 allowed_special 告诉它："这是个特殊 token，请保留整体，不要拆开"。
integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})

print(integers)

[15496, 11, 466, 345, 588, 8887, 30, 220, 50256, 554, 262, 4252, 18250, 8812, 2114, 1659, 617, 34680, 27271, 13]
```

```
In [15]: strs = tokenizer.decode(integers)
print(strs)

Hello, do you like tea? <|endoftext|> In the sunlit terracesof someunknownPlace.
```

BPE 是怎么做到的？—— 直观感受

先用刚才让 V2 抓瞎的那段生僻词文本，看看 BPE 的表现：

```
In [ ]: # 用 BPE tokenizer 处理同样的生僻词文本
bpe_ids = tokenizer.encode(text_with_rare_words)
bpe_decoded = tokenizer.decode(bpe_ids)

print("原文：")
print(text_with_rare_words)
print("\nBPE encode → decode 结果：")
print(bpe_decoded)

# 逐个展示 BPE 是怎么拆词的
print("\n每个 token 的对应关系：")
for tid in bpe_ids:
    token_str = tokenizer.decode([tid])
    print(f" {tid:>6} → '{token_str}'")
```

看到了吗？pneumonoultramicroscopicsilicovolcanoconiosis 这种超长医学词，BPE 把它拆成了 p neu mon oul tra micro scop ic sil ico vol cano con iosis 等一连串小片段。quasifibrillators 也被拆成了 quasi fib rill ators。

没有一个 <|unk|>！因为 BPE 不要求认识完整的词——它只需要认识组成这些词的“零件”。就好像你不认识“三碘甲状腺原氨酸”这个化学名词，但你认识“三”“碘”“甲状腺”“原氨酸”，就能读出来。

BPE 的核心原理

BPE 的思路简单得出奇——“哪两个 token 老是一起出现，就把它合并成一个”。算法步骤：

1. **从字符级开始**：初始词表只包含每个独立的字符（包括空格、标点、Unicode 字符等）

2. 统计所有相邻 token 对的出现频率
3. 把出现次数最多的一对合并成新的 token，加入词表
4. 重复 2-3，直到词表达到指定大小（GPT-2 是 50,257 个）

举个例子。假设训练文本只有这一句话： "low low low low lower"：

```
初始: l o w _ l o w _ l o w _ l o w _ l o w e r    (_ 代表空格)
高频对: "l"+"o" → "lo"
合并: l o w _ l o w _ l o w _ l o w _ l o w e r
高频对: "lo"+"w" → "low"
合并: l o w _ l o w _ l o w _ l o w _ l o w e r
高频对: "low"+"_" → "low_"
合并: l o w _ l o w _ l o w _ l o w _ l o w e r
高频对: "low_"+"low_" → 出现频率还不够高，换 "e"+"r" → "er"
合并: l o w _ l o w _ l o w _ l o w _ l o w e r
...
```

真正的 BPE 在更大的文本上跑几万次合并，最终的词表里既有单个字符（a, b, !, ?），也有常见子词（ing, tion, the, self），还有高频完整词（the, and, a）。遇到生词就往下拆，拆到词表里有的子词为止。

GPT-2 的 BPE 细节

GPT-2 用的是**字节级 BPE**（Byte-level BPE），比刚才例子多两个关键设计：

- **以字节为单位**，不是以 Unicode 字符为单位——所以它理论上能编码任何语言、任何符号，甚至 emoji。词表大小 50,257。
- **只有一个特殊 token**： <|endoftext|>（ID = 50256）。不需要 <|unk|>（因为 BPE 不会遇到真正的"未知字符"），也不单独区分 BOS/EOS/PAD——全部复用 <|endoftext|>。

总结一句话：**V1/V2 是"背词典"，BPE 是"学词根"**。背词典遇到生词就傻眼，学词根可以拼出任何词。这就是为什么所有主流 LLM 都在用 BPE 或它的变体。

在第 5 章我们会从零实现一个 BPE tokenizer，把上面的合并循环写成代码。现在先用 tiktoken 现成的，把注意力放在"处理完 token 之后做什么"。

第三步：从 token 序列到训练样本——"根据上文预测下一个词"

好，现在我们有办法把任意文本变成一串 token ID 了。那模型拿到这串 ID 之后，到底要做什么？

答案出奇的简单——**猜下一个 token**。

把 LLM 的训练过程想象成“填字游戏”：

输入（已知上文）		模型要预测的（下一个 token）
[LLMs]	→	learn ← 看到 "LLMs", 猜 "learn"
[LLMs, learn]	→	to ← 看到 "LLMs learn", 猜 "to"
[LLMs, learn, to]	→	predict ← 看到 "LLMs learn to", 猜 "predict"
[LLMs, learn, to, predict]	→ next	← ...
...		

这就是大模型唯一在做的事：给定一串 token，猜接下来最可能是哪个 token。

训练时，正确答案（真实文本的下一个 token）是已知的——模型不断调整内部参数，让自己猜的越来越准。

上图的每一对（上文，下一个）就是一条训练样本。构造方法很简单：**把真实文本右移一位，作为“标准答案”**。模型看左边的 token 序列，输出对下一个 token 的预测，然后跟右边的“标准答案”对比，算出差多少（loss），反向传播修正参数。

下面动手把《The Verdict》变成这样的训练样本。

```
In [16]: # the-verdict.txt 文件内就是一段文章，我们可以用Tokenizer，生成一系列的tokenId
raw_text = DATA_PATH.read_text(encoding="utf-8")

enc_text = tokenizer.encode(raw_text) # 将原文转为一个tokenId的序列
print(len(enc_text))
```

5145

举个例子，说明一下训练的数据长什么样：

```
In [17]: # 取出一段
enc_sample = enc_text[50:]
print(enc_sample[:10])
```

[290, 4920, 2241, 287, 257, 4489, 64, 319, 262, 34686]

```
In [18]: context_size = 4 # 设置每个窗口最大为4

x = enc_sample[:context_size]
y = enc_sample[1:context_size+1]
print("假设x我们给大模型的输入，我们要想让大模型能预测下一个词，那输出就是x向右移一位")
print(f"x: {x}")
print(f"y: {y}")
```

假设x我们给大模型的输入，我们要想让大模型能预测下一个词，那输出就是x向右移一位，即y：

x: [290, 4920, 2241, 287]

y: [4920, 2241, 287, 257]

预测过程就像是这样：

```
In [19]: for i in range(1, context_size+1):
context = enc_sample[:i]
```

```
desired = enc_sample[i]

print(context, "---->", desired)
```

```
[290] ----> 4920
[290, 4920] ----> 2241
[290, 4920, 2241] ----> 287
[290, 4920, 2241, 287] ----> 257
```

用文本表示，过程也就是这样：

```
In [20]: for i in range(1, context_size+1):
        context = enc_sample[:i]
        desired = enc_sample[i]

        print(tokenizer.decode(context), "---->", tokenizer.decode([desired]))
```

```
and ----> established
and established ----> himself
and established himself ----> in
and established himself in ----> a
```

下面我们做一个简单的数据加载器，把原始数据转变为可供于训练的格式（input & target）

```
In [21]: # 先导入torch库
import torch
print("PyTorch version:", torch.__version__)
```

PyTorch version: 2.12.0+cpu

在动手之前，先认识一下 PyTorch 里的两个“搬运工”：

- **Dataset**：你可以把它理解成“数据仓库”。我们要做的 `GPTDatasetV1` 继承它，重写 `__len__`（告诉别人仓库里有多少条数据）和 `__getitem__`（按索引取出一条数据）。它只管“单条存取”，不管怎么打包。
- **DataLoader**：你可以把它理解成“搬运工”。它接收一个 Dataset，按 `batch_size` 把单条数据摞成一个 batch，还能自动 shuffle（打乱顺序）、多线程加载（`num_workers`）、丢弃最后不够一个 batch 的零头（`drop_last`）。

总结：**Dataset 管“存”，DataLoader 管“搬”**。下面我们把两个一起写出来。

```
In [ ]: # 我们用滑动窗口的方法
        # 例如：假设context长度为4，
        # 原始数据    = [in, the, heart, of, the, city]
        # input  (x)  = [in, the, heart, of], x长度为4
        # target (y)  = [the, heart, of, the], y为x向右移一位

        # 代码实现
from torch.utils.data import Dataset, DataLoader
import torch

class GPTDatasetV1(Dataset):
    def __init__(
        self,
        txt,
```

```

tokenizer, # 分词器 (这里用的是 tiktoken 的 GPT-2 BPE 分词器)
max_length, # 每个窗口的 token 数
stride, # 步长, 用于控制紧挨的两个窗口之间的间隔, 比如步长为2, 则[1, 2,
):
    self.input_ids = []
    self.target_ids = []

    # 将文本通过分词器处理, 得到训练数据的tokenId序列
    # allowed_special: 告诉 tiktoken "<|endoftext|>" 是特殊 token, 保留整体才
    # 如果是单篇纯文本 (比如本例的《The Verdict》), 文本里没有这个符号, 传不
    # 但如果训练数据是多篇文章拼接的, 中间会有 <|endoftext|> 做分隔, 就必须加
    token_ids = tokenizer.encode(txt, allowed_special={"<|endoftext|>"})
    assert len(token_ids) > max_length, "token_id的数量至少要大于max_length"

    # 用滑动窗口将全文进行切分
    # 比如正文为: [1, 2, 3, 4, 5, 6, 7, 8, 9], max_length = 4, stride = 2
    # 则
    # 第一轮: input_chunk = [1, 2, 3, 4], target_chunk = [2, 3, 4, 5]
    # 第二轮: input_chunk = [3, 4, 5, 6], target_chunk = [4, 5, 6, 7]
    # ...
    # self.input_ids = [[1, 2, 3, 4], [3, 4, 5, 6]...]
    #               预测↓           预测↓
    # self.target_ids = [[2, 3, 4, 5], [4, 5, 6, 7]...]
    for i in range(0, len(token_ids) - max_length, stride):
        input_chunk = token_ids[i : i + max_length]
        target_chunk = token_ids[i + 1 : i + max_length + 1]
        self.input_ids.append(torch.tensor(input_chunk))
        self.target_ids.append(torch.tensor(target_chunk))

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, index) -> tuple:
        return self.input_ids[index], self.target_ids[index]

def create_dataloader_v1(txt, batch_size=4, max_length=256,
                        stride=128, shuffle=True, drop_last=True,
                        num_workers=0, tokenizer=None):

    # 如果调用方没传 tokenizer, 就自己建一个 (复用外部 tokenizer 可以避免重复初始
    if tokenizer is None:
        tokenizer = tiktoken.get_encoding("gpt2")

    # Create dataset
    dataset = GPTDatasetV1(txt, tokenizer, max_length, stride)

    # Create dataloader
    dataloader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        drop_last=drop_last,
        num_workers=num_workers
    )

    return dataloader

```

下面我们用导入器试试处理一下, 是否能得到想要的训练样本

```
In [23]: raw_text = DATA_PATH.read_text(encoding="utf-8")

tokenizer = tiktoken.get_encoding("gpt2")
encoded_str = tokenizer.encode(raw_text)
print("原始数据前20位的tokenID为", encoded_str[:20])

# 按批大小为1, 窗口大小为4, 步长为1创建dataLoader
dataloader = create_dataloader_v1(
    raw_text, batch_size=1, max_length=4, stride=1, shuffle=False
)
```

原始数据前20位的tokenID为 [40, 367, 2885, 1464, 1807, 3619, 402, 271, 10899, 2138, 257, 7026, 15632, 438, 2016, 257, 922, 5891, 1576, 438]

根据执行结果原始数据转为tokenID, 前20位是

[40, 367, 2885, 1464, 1807, 3619, 402, 271, 10899, 2138, 257, 7026, 15632, 438, 2016, 257, 922, 5891, 1576, 438]

我们根据原始数据前20位和dataloader的参数, 可以先推断一下生成的数据格式

第一批: input_ids = [40, 367, 2885, 1464], target_ids = [367, 2885, 1464, 1807]

第二批: input_ids = [367, 2885, 1464, 1807], target_ids = [2885, 1464, 1807, 3619]

```
In [24]: data_iter = iter(dataloader)
first_batch = next(data_iter)
print("第一批数据", first_batch)

second_batch = next(data_iter) # 通过不断next(data_iter)获得下一批
print("第二批数据", second_batch)
```

第一批数据 [tensor([[40, 367, 2885, 1464]]), tensor([[367, 2885, 1464, 1807]])]

第二批数据 [tensor([[367, 2885, 1464, 1807]]), tensor([[2885, 1464, 1807, 3619]])]

我们还可以用 dataloader 创建批量输出, 增大步幅。

关于 stride 的选择: stride 越小, 切出来的样本越多, 但相邻样本的输入高度重叠 (只差一个 token 的位置), 相当于模型把同一条文本翻来覆去背了很多遍——浪费算力, 训练出来的模型泛化能力也差。一般把 stride 设成和 max_length 一样大 (样本无重叠), 或者 max_length 的一半左右。

例如: 我们创建批大小为 8, 步幅为 4, 窗口为 4 的 dataloader, 看看它生成的数据是不是符合我们预期的那样:

```
In [25]: dataloader = create_dataloader_v1(raw_text, batch_size = 8, max_length=4, stride
data_iter = iter(dataloader)
inputs, targets = next(data_iter)
print("Inputs:\n", inputs)
print("\nTargets:\n", targets)
```

```
Inputs:
tensor([[ 40,   367,  2885,  1464],
        [ 1807,  3619,   402,   271],
        [10899,  2138,   257,  7026],
        [15632,   438,  2016,   257],
        [  922,  5891,  1576,   438],
        [  568,   340,   373,   645],
        [ 1049,  5975,   284,   502],
        [  284,  3285,   326,    11]])
```

```
Targets:
tensor([[ 367,  2885,  1464,  1807],
        [ 3619,   402,   271, 10899],
        [ 2138,   257,  7026, 15632],
        [  438,  2016,   257,   922],
        [ 5891,  1576,   438,   568],
        [  340,   373,   645,  1049],
        [ 5975,   284,   502,   284],
        [ 3285,   326,    11,   287]])
```

至此，我们彻底搞懂了Tokenizer和处理原始数据的方法。也就是模型输入的第一步即：我们要让模型读文本，先通过Tokenizer将文本转为tokenID序列

第四步：从整数到向量——让数字有"含义"

前面 dataloader 吐出来的 `inputs`，形状是 `[batch_size, max_length]`，里面全是整数。比如 `[40, 367, 2885, 1464]`。

但这里有一个很容易忽略的问题：**token ID 只是个编号，不是语义**。ID 40 跟 ID 367 挨着，不代表这两个词意思相近——它们只是碰巧排在词表第 40 行和第 367 行而已。

所以接下来做两件事：

1. **Token Embedding**：给每个 token ID 配一个"含义向量"
2. **Position Embedding**：给每个位置也配一个向量，告诉模型"这是第几个token"

两者相加，才是喂给 模型 的最终输入。

4.1 Token Embedding —— 一张"可训练的查找表"

先用一个极小的例子感受一下。假设我们词表中有 7 个 token，想让每个 token 用 3 个数字来表示（现实中是 256~几千维）。

做法很简单：初始化一张 7×3 的随机数字表，第 i 行就是 token i 的向量。训练时模型会不断调整这些数字，达到训练的效果：

```
In [26]: import torch

torch.manual_seed(123)

vocab_size = 7
output_dim = 3

embedding_layer = torch.nn.Embedding(vocab_size, output_dim)
```

```
print("embedding 矩阵形状:", embedding_layer.weight.shape)
print(embedding_layer.weight)
```

```
embedding 矩阵形状: torch.Size([7, 3])
Parameter containing:
tensor([[ 0.3374, -0.1778, -0.3035],
        [-0.5880,  0.3486,  1.3010],
        [ 1.2753, -0.2010, -0.1606],
        [-0.4015,  0.6957, -1.8061],
        [ 1.8960,  0.3255, -0.6315],
        [-2.8400, -0.7849, -1.4096],
        [-0.4076,  0.7953,  0.9985]], requires_grad=True)
```

现在假设有一句话被 tokenizer 转成了 [2, 3, 5, 1]。embedding 做的事情就是——去那张表里查第 2、3、5、1 行，取出来摞在一起：

```
In [27]: input_ids = torch.tensor([2, 3, 5, 1])

print("第 3 行的向量:")
print(embedding_layer(torch.tensor([3])))

print("\ninput_ids 对应的 embedding:")
print(embedding_layer(input_ids))

print("\n输出形状:", embedding_layer(input_ids).shape)
```

```
第 3 行的向量:
tensor([[ -0.4015,  0.6957, -1.8061]], grad_fn=<EmbeddingBackward0>)
```

```
input_ids 对应的 embedding:
tensor([[ 1.2753, -0.2010, -0.1606],
        [-0.4015,  0.6957, -1.8061],
        [-2.8400, -0.7849, -1.4096],
        [-0.5880,  0.3486,  1.3010]], grad_fn=<EmbeddingBackward0>)
```

```
输出形状: torch.Size([4, 3])
```

总结：embedding 不是对 ID 做数学运算，而是拿 ID 当索引去“查表”。这张表一开始是随机的，训练中会被反向传播不断打磨，让语义相近的词向量也相近。

4.2 接上真实的 dataloader

刚才例子只有 7 个 token。回到实际场景：GPT-2 词表大小 50,257，前面 dataloader 给我们的 inputs 形状是 [8, 4] (batch_size=8, max_length=4)。

给每个 token 配一个 256 维的向量：

这里为了演示方便用了 256 维。真正的 GPT-2 各型号 embedding 维度分别是：Small 768、Medium 1024、Large 1280、XL 1600。维度越高，模型能编码的语义信息越丰富，但参数数量和算力也成倍涨。

```
In [28]: vocab_size = 50257      # GPT-2 tokenizer 的词表大小
         output_dim = 256      # 每个 token 映射成 256 维向量
```

```
# inputs 来自上一节的 dataloader, 形状是 [batch_size, max_length]
token_embedding_layer = torch.nn.Embedding(vocab_size, output_dim)
token_embeddings = token_embedding_layer(inputs)

print("inputs shape:", inputs.shape)
print("token_embeddings shape:", token_embeddings.shape)
```

inputs shape: torch.Size([8, 4])

token_embeddings shape: torch.Size([8, 4, 256])

输出 [8, 4, 256], 拆开看:

- 8: 一个 batch 里 8 条样本
- 4: 每条样本 4 个 token
- 256: 每个 token 现在是 256 维向量

4.3 还缺什么? ——位置信息

Token embedding 有一个盲区: 同一个词不管出现在第 0 个位置还是第 3 个位置, 查出来的向量一模一样。

但顺序很重要。举个中文例子:

"我喜欢你" → 主语是"我"

"你喜欢我" → 主语是"你"

字完全一样, 顺序不同, 意思完全相反。如果模型不知道位置, "我喜欢你"和"你喜欢我"在它眼里是同一堆 token ID 的乱序排列——这肯定不行。

所以 GPT 的做法很简单: 跟 token embedding 一样, 再建一张"位置查找表"。第 0 个位置有一行向量, 第 1 个位置有另一行.....位置向量也是随机初始化、训练中学习的。

先用 4 个位置、3 维向量的小例子感受一下:

```
In [29]: # 4 个位置, 每个位置用 3 维向量表示 (跟上面的 token embedding 对应)
max_length = 4
output_dim = 3

torch.manual_seed(456)
pos_layer = torch.nn.Embedding(max_length, output_dim)

# 位置 0, 1, 2, 3 各自查表得到一个向量
pos_ids = torch.arange(max_length)
pos_embeddings = pos_layer(pos_ids)

print("位置 ID:", pos_ids.tolist())
print("位置向量 shape:", pos_embeddings.shape)
print(pos_embeddings)
```

位置 ID: [0, 1, 2, 3]

位置向量 shape: torch.Size([4, 3])

```
tensor([[ -2.4306,  0.7544,  1.6025],
        [-1.1431, -0.1645,  0.6943],
        [ 1.2980, -0.7403,  1.0322],
        [-0.7191, -0.0373,  0.2765]], grad_fn=<EmbeddingBackward0>)
```


位置 0、1、2、3 各有自己的一行向量，互不相同。这样一来，同一个词出现在"位置 0"和"位置 3"时，embedding 是一样的，但加上不同的位置向量后，最终结果就不同了——模型就能区分"我喜欢你"和"你喜欢我"。

回到真实场景，GPT-2 用 256 维：

```
In [30]: max_length = 4
         output_dim = 256

         position_embedding_layer = torch.nn.Embedding(max_length, output_dim)
         position_ids = torch.arange(max_length)
         position_embeddings = position_embedding_layer(position_ids)

         print("position_ids:", position_ids)
         print("position_embeddings shape:", position_embeddings.shape)
```

```
position_ids: tensor([0, 1, 2, 3])
position_embeddings shape: torch.Size([4, 256])
```

`position_embeddings` 形状是 `[4, 256]` ——4 个位置各有 256 维向量。把它加到 `token_embeddings` 上：

```
token_embeddings:   [8, 4, 256]   ← 8 条样本各自的 token 含义
position_embeddings: [4, 256]   ← 4 个位置各自的位置信息
                   相加 ↓
input_embeddings:   [8, 4, 256]   ← 最终输入
PyTorch 会自动把 [4, 256] 广播到 batch 里每一条样本上：
```

```
In [31]: input_embeddings = token_embeddings + position_embeddings

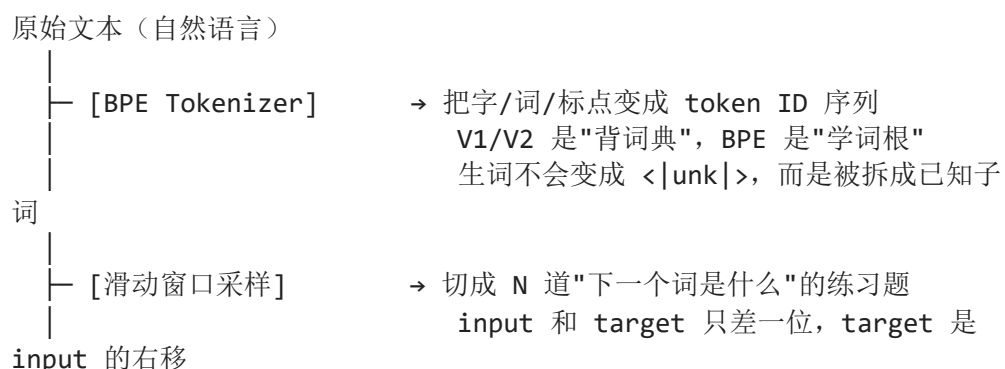
         print("token_embeddings shape:", token_embeddings.shape)
         print("position_embeddings shape:", position_embeddings.shape)
         print("input_embeddings shape:", input_embeddings.shape)
```

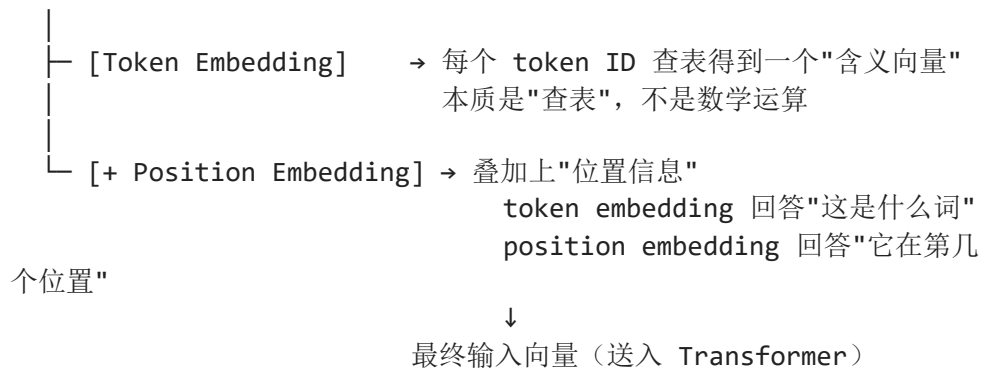
```
token_embeddings shape: torch.Size([8, 4, 256])
position_embeddings shape: torch.Size([4, 256])
input_embeddings shape: torch.Size([8, 4, 256])
```

这样，最终输入就准备好了。

本章回顾

翻回开头看那张**数据流全景图**，现在每一行你应该都能说清楚它做了什么：





本章的核心就一句话：**把人类能读的字符串，一步步变成模型能算的张量。** 每一步做了什么、为什么需要它，就是整章要讲的东西。

到此，数据已经从 "I HAD always thought..." 变成了 [8, 4, 256] 的张量。下一步（第 3 章），把这些向量送进 Attention 机制——让模型学会"读"这些向量之间的关系。